

P1099R5

Using Enum

Published Proposal, 2019-07-16

This version:

<https://github.com/atomgalaxy/using-enum/using-enum.bs>

Authors:

Gašper Ažman <gasper.azman@gmail.com>

Jonathan Müller <jonathan.mueller@foonathan.net>

Audience:

CWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

Class enums are restricted namespaces. Let's extend the using declaration to them.

Table of Contents

- 1 **Revision History**
- 2 **Status of this paper**
- 3 **Motivation**
- 4 **Rationale**
 - 4.1 Consistency
 - 4.2 Better Identifiers
 - 4.3 Evidence of Need
- 5 **Proposal**

5.1 Syntax: `using ENUM_ID::IDENTIFIER`

5.2 Syntax: `using enum IDENTIFIER`

6 Examples

6.1 Strongly typed enums with global identifiers

6.2 Switching with no syntax overhead

6.3 Adding ADL-only Functions to Enumerations:

7 Frequently Asked Questions

7.1 Has this been implemented?

7.2 Can I do this with unscoped enums?

7.3 Are you proposing mirroring the namespace alias syntax as well?

7.4 Why not allow `using enum struct/class ENUM_ID; ?`

7.5 Why propose `using ENUM_ID::IDENTIFIER` at all?

8 Clarifications for Special Cases

8.1 [namespace.udecl] p3

8.2 [namespace.udecl] p8

8.3 Commas in the using declaration

8.4 Commas in the *using-enum-declaration*

8.5 Names *using-enum-declaration* introduces

9 Proposed Wording

9.1 Preface

9.2 Changes

10 Acknowledgements

References

Informative References

§ 1. Revision History

- r1: Typos. Changed target from LEWG to EWG.
- r2: Added FAQ and wording, and extended to regular enums.
- r3: Added examples. Nothing of substance changed.

- r4: Clarified that `using enum enum_name` is supposed to have semantics as if the names were declared in the local scope as opposed to the way *using-directive* does it, as per EWG poll. Added Eric Niebler's example for using this feature to enable ADL-only functions on exposed enumerators (to support `std::begin` being a CPO). Added editorial notes for renaming the current Using Directive to Using Namespace Directive on Richard Smith's request.
- r5: Applied fixes from Jens Maurer: added feature test macro, a number of technical wording fixes, added example, renamed from *using-enum-directive* to *using-enum-declaration*. Removed editorial notes for renaming *using-namespace-directive* since they are now not needed.
 - r5, post round 1 of CWG review: added clashing example, added "completely specified" wording to `[dcl.enum]`, added wording that named enum is completely-specified.
 - r5, post round 1.5 of CWG review: used wording from CWG, split example in `[enum.udecl]`.
 - r5, post round 2 of CWG review: added example in `namespace.udir`.
 - r5, morning: fixed reference to N4800, added 'Example' to second example in note.
 - r5, post round 3 of CWG review: removed full stops at end of code comments in examples, 'orange, apple' -> 'orange and apple', added [Example: --end example] to both examples in the 2nd note to make it clear where they begin and end, 'are in conflict' -> 'conflict'.
 - r5: Approved by CWG.

§ 2. Status of this paper

This paper has been approved by CWG in Cologne 2019 for C++20 after being approved by EWG in Kona 2019 (ship vehicle C++20).

§ 3. Motivation

The single biggest deterrent to use of scoped enumerations is the inability to associate them with a *using directive*.

— Dan Saks

Consider an enum class:

```
enum class rgba_color_channel { red, green, blue, alpha };
```

Currently, a switch using this enum looks as follows:

```
std::string_view to_string(rgba_color_channel channel) {  
    switch (channel) {  
        case rgba_color_channel::red:    return "red";  
        case rgba_color_channel::green:  return "green";  
        case rgba_color_channel::blue:   return "blue";  
        case rgba_color_channel::alpha:  return "alpha";  
    }  
}
```

The necessary repetition of the `enum class` name reduces legibility by introducing noise in contexts where said name is obvious.

To eliminate the noise penalty for introducing long (but descriptive) `enum class` names, this paper proposes that the statement

```
using enum rgba_color_channel;
```

introduce the enumerator identifiers into the local scope, so they may be referred to unqualified.

Furthermore, the syntax

```
using rgba_color_channel::red;
```

should bring the identifier `red` into the local scope, so it may be used unqualified.

The above example would then be written as

```
std::string_view to_string(rgba_color_channel channel) {  
    switch (my_channel) {  
        using enum rgba_color_channel;  
        case red:    return "red";  
        case green:  return "green";  
        case blue:   return "blue";  
        case alpha:  return "alpha";  
    }  
}
```

§ 4. Rationale

§ 4.1. Consistency

`enum class`s and `enums` are not classes - they are closer to namespaces comprising `static constexpr` inline variables. The familiar `using` syntax that works for namespaces should therefore apply to them as well, in some fashion. Because they are closed, small, and do not contain overload sets, we can do better than the *using-directive* does for namespaces, and actually get the identifiers into the local scope, which is what the user expects.

§ 4.2. Better Identifiers

The introduction of this feature would allow better naming of enumerations. Currently, enums are named with as short an identifier as possible, often to the point of absurdity, when they are reduced to completely nondescriptive abbreviations that only hint at their proper meaning. (Just what does `zfqc::add_op` really mean?)

With this feature, identifiers become available to unqualified lookup in local contexts where their source is obvious, giving control of lookup style back to the user of the enum, instead of baking lookup semantics into the type of the enum.

§ 4.3. Evidence of Need

At a casual search, we were able to locate this [thread on stackoverflow](#).

Anecdotally, 100% of people the authors have shown this to (~30) at CppCon have displayed a very enthusiastic response, with frequent comments of "I'd use enum classes but they are too verbose, this solves my problem!"

§ 5. Proposal

§ 5.1. Syntax: `using ENUM_ID::IDENTIFIER`

We propose to allow the syntax of

```
using ENUM_ID::IDENTIFIER
```

to introduce the `IDENTIFIER` into the local namespace, aliasing `ENUM_ID::IDENTIFIER`.

This would mirror the current syntax for introducing namespaced names into the current scope.

Note: this does not conflict with [\[P0945R0\]](#), because that paper only deals with the syntax `using name = id-expression`, which duplicates the enumerator name.

§ 5.2. Syntax: `using enum IDENTIFIER`

We propose the addition of a new `using enum` statement:

```
using enum IDENTIFIER;
```

This makes all the enumerators of the enum available for lookup in the local scope. It's almost as if it expanded to a series of `using ENUM::ENUMERATOR` statements for every enumerator in the enum, but doesn't actually introduce any declarations into the local scope.

(Note: this was changed from "works as a using-directive" to the current way with a strong direction poll from EWG.)

§ 6. Examples

§ 6.1. Strongly typed enums with global identifiers

This proposal lets you make strongly-typed enums still export their identifiers to namespace scope, therefore behaving like the old enums in that respect:

```

namespace my_lib {

enum class errcode {
    SUCCESS = 0,
    ENOMEM = 1,
    EAGAIN = 2,
    ET00SLOW = 3
};

using enum errcode; // import enumerators into namespace

}

namespace NS {

my_lib::errcode get_widget() {
    using namespace my_lib;
    return ET00SLOW; // works, and conversions to int don't.
}

}

```

§ 6.2. Switching with no syntax overhead

The proposal allows for importing enums inside the switch body, which is a scope, and using them for labels:

```

enum class rgba_color_channel { red, green, blue, alpha};

std::string_view to_string(rgba_color_channel channel) {
    switch (my_channel) {
        using enum rgba_color_channel;
        case red:    return "red";
        case green:  return "green";
        case blue:   return "blue";
        case alpha:  return "alpha";
    }
}

```

§ 6.3. Adding ADL-only Functions to Enumerations:

The proposal allows for adding ADL-only functions to enumerations without enumerators (supported now) and enumerators (currently not supported):

```
namespace ns {
    struct E_detail {
        enum E {
            e1
        };
        friend void swap(E&, E&); // adl-only swap in the only associated scope
    };
    using E = E_detail::E; // import E into ns
    using enum E;           // expose the enumerators of E in ns. Also note that
}

int main() {
    auto x = ns::e1;
    auto y = ns::e2;
    swap(x, y); // finds the swap in the associated struct
}
```

This example was slightly modified from Eric Niebler's on the *lib* mailing list when trying to find a way to make `std::begin` and `std::end` CPOs in a backwards-compatible fashion.

§ 7. Frequently Asked Questions

§ 7.1. Has this been implemented?

Yes. The author has an implementation in clang. It has not been reviewed or released yet, however. There do not seem to be major issues with implementation. In particular, the `using ENUM :: IDENTIFIER` syntax literally entailed removing a condition from an if-statement, and that was it.

§ 7.2. Can I do this with unscoped enums?

Yes. The motivation for that is the pattern


```
class foo {
    enum bar {
        A,
        B,
        C
    };
};
```

which was superseded by scoped enums. With the feature this paper proposes one can bring `A`, `B` and `C` into the local scope by invoking:

```
using enum ::foo::bar;
```

§ 7.3. Are you proposing mirroring the namespace alias syntax as well?

No. We already have a way to do that, and it looks like this:

```
using my_alias = my::name_space::enum_name;
```

In addition, [\[P0945R0\]](#) proposes deprecating namespace aliases in favor of generalized `using name = id_expression`, so doing this would go counter the current movement of the standard.

§ 7.4. Why not allow `using enum struct/class ENUM_ID`?

Because would have been a needless complication and would introduce another layer of "struct and class don't match" linter errors that current `classes` and `structs` already have with forward declarations.

§ 7.5. Why propose `using ENUM_ID::IDENTIFIER` at all?

... given that the following already works:

```
constexpr auto red = rgba_color_channel::red;
```

and that, given [\[P0945R0\]](#), this will work:

```
using red = rgba_color_channel::red;
```

The reason is "DRY" - don't repeat yourself - one is forced to repeat the name of the enumerator. That said, the authors are perfectly willing to throw this part of the paper out if the `using enum ENUM_ID` piece gets consensus and this is the stumbling block.

§ 8. Clarifications for Special Cases

This section lists additional clarifications that may help inform the the wording.

§ 8.1. [namespace.udecl] p3

Interplays with [namespace.udecl] p10:

```
enum E { x };
struct S {
    enum H { y };
    enum class K { z };
    using E::x; // OK, introduces x into S
    using E::x; // error, redeclaration in class scope
    using H::y; // error, redeclaration in class scope
    using K::z; // OK, introduces z into S
};
```

In declarative regions which do allow multiple declarations, however:

```
enum E { x };
namespace NS {
    enum H { y };
    enum class K { z };
    using E::x; // OK, introduces x into NS
    using E::x; // OK, just a redeclaration of the same entity
    using H::y; // OK, redeclaration of the same entity
    using K::z; // OK, introduces z into NS
};
```

§ 8.2. [namespace.udecl] p8

This change is meant to allow the introduction of class members that are enumerators into non-class scope. Consider this example:

```
struct S {  
    enum E { x };  
    enum class EC { y };  
    using EC::y;  
};  
  
void f() {  
    using S::x; // OK  
    x; // resolves to S::E::x;  
    using S::y; // OK  
    y; // resolves to S::EC::y;  
}
```

§ 8.3. Commas in the using declaration

Since the grammar of the using declaration is not changed, it is valid to import multiple enumerators at the same time:

```
enum class E { a, b, c };  
using E::a, E::b, E::c; // OK, imports all three
```

§ 8.4. Commas in the *using-enum-declaration*

Since "using namespace" does not allow them, this paper did not propose allowing the listing of several enumeration names in the *using-enum-declaration*.

§ 8.5. Names *using-enum-declaration* introduces

The *using-enum-declaration* only introduces the names of the enumerators into the declarative region where it appears. It does not introduce the name of the enumeration it names.

Example:

```

struct B {
    enum class E { x };
};
enum class H { y };
struct C : B {
    using enum B::E; // OK, introduces E::x into C
    using enum H; // OK, introduces y into C. Does not introduce H
};

```

§ 9. Proposed Wording

§ 9.1. Preface

The idea is that the identifiers appear as if they were declared in the declarative region where the *using-enum-declaration* appears, and not model the using-directive's "enclosing namespace" wording.

All wording is relative to the working draft of the ISO/IEC IS 14882: N4765, though, as it is almost entirely additive, it is also a valid diff to N4800.

§ 9.2. Changes

Under [namespace.udecl]:

3. In a *using-declaration* used as a *member-declaration*, each *using-declarator* ~~is~~ shall either name an enumerator or have a *nested-name-specifier* ~~shall name~~ naming a base class of the class being defined. *[Example:*

```

enum class button { up, down };
struct S {
    using button::up;
    button b = up; // OK
};

```

-- end example]

~~7. A using-declaration shall not name a scoped enumerator.~~

8. A *using-declaration* that names a class member **other than an enumerator** shall be a *member-declaration*.

Under [dcl.dcl], in [dcl.enum], add subclause titled "Using Enum Declaration", with the stable reference "[enum.udecl]".

using-enum-declaration:

using *elaborated-enum-specifier* ;

1. The *elaborated-enum-specifier* shall not name a dependent type and the type shall have a reachable *enum-specifier*.
2. A *using-enum-declaration* introduces the enumerator names of the named enumeration as if by a *using-declaration* for each enumerator.

[*Note:* A *using-enum-declaration* in class scope adds the enumerators of the named enumeration as members to the scope. This means they are accessible for member lookup. [*Example:*

```
enum class fruit { orange, apple };
struct S {
    using enum fruit; // OK, introduces orange and apple into S
};
void f() {
    S s;
    s.orange; // OK, names fruit::orange
    S::orange; // OK, names fruit::orange
}
```

-- end example]

Two *using-enum-declarations* that introduce two enumerators of the same name conflict. [*Example:*

```
enum class fruit { orange, apple };
enum class color { red, orange };
void f() {
    using enum fruit; // OK
    using enum color; // error, color::orange and fruit::orange conflict
}
```

-- end example] -- end note]

Under [basic.def], add (just after *using-directive*) (and renumber section):

2.17. — it is a *using-enum-declaration*

In [dcl.dcl], under *block-declaration*:

block-declaration

[...]

using-declaration

using-enum-declaration

In [class.mem], under *member-declaration*:

member-declaration

[...]

using-declaration

using-enum-declaration

In [dcl.type.elab]:

elaborated-type-specifier:

[...]

~~enum nested-name-specifier_{opt} identifier~~

elaborated-enum-specifier

elaborated-enum-specifier:

enum *nested-name-specifier*_{opt} identifier

To table 17 (Feature-test macros), add the feature test macro:

Name	Value
<code>_cpp_using_enum</code>	PLACEHOLDER DATE

The PLACEHOLDER DATE should be replaced with the appropriate integral constant of type long.

§ 10. Acknowledgements

(Alphabetically, by first name)

- Barry Revzin: feedback on R0
- BSI C++ WG: feedback throughout the life of the process
- Casey Carter: realization that this proposal allows adding adl-only functions to enumerations, typography feedback.

- CWG for their help with wording
- Dan Saks: early feedback, encouragement, quotation at start
- Eric Niebler: for the example of adl-only functions for enumerations.
- Graham Haynes: feedback on R0, and suggesting examples
- Jeff Snyder: feedback, championing with EWG
- Jens Maurer: wording feedback, the change to *using-enum-declaration*, and the encouragement to add the feature-test macro and split out the *elaborated-enum-specifier* from the other *elaborated-type-specifiers*, edge cases.
- Lisa Lippincott: early feedback on preprint.
- Marcel Ebmer: early feedback on preprint.
- Richard Smith: change design to direct injection of names, why enum names must not be dependent types, and a lot of help with wording. The initial wording without Richard's involvement would have been much, much worse.
- Roger Orr: championing with CWG, and the button example in namespace.udir
- Tomas Puerle: encouragement to extend to regular `enums`, and other feedback

§ References

§ Informative References

[P0945R0]

Richard Smith. [Generalizing alias declarations](https://wg21.link/p0945r0). 10 February 2018. URL: <https://wg21.link/p0945r0>